GOVERNMENT JOB TRACKER & ALERT SYSTEM

Data Structures & Algorithms Project Report

Submitted By:
Kriti Pandey (12407789)

Under the Guidance of:
Mam Prerita

Course: CSE443

School of Computer Science and Engineering

LOVELY PROFESSIONAL UNIVERSITY

Abstract

Government job recruitment in India is spread across many independent boards - SSC, UPSC, State Public Service Commissions, Railways, banking bodies, and defence services - each publishing notifications through its own portal. Aspiring candidates must manually track multiple websites, and a missed deadline typically means waiting for the next recruitment cycle. The Government Job Tracker & Alert System addresses this challenge by organizing government job postings into a centralized system that supports efficient searching, sorting, deadline-based filtering, and alerts for approaching application deadlines.

This report documents the implementation of the Government Job Tracker & Alert System, in which core features such as keyword search, deadline ordering, range filtering, duplicate detection, and alerting are implemented using dedicated data structures and algorithms. A Trie powers prefix search; a binary Min-Heap maintains a deadline-priority queue; a hand-written Merge Sort orders postings chronologically; Binary Search answers “closing within N days” queries efficiently; and hashing provides duplicate detection and O(1) average-case lookup. The report covers the problem definition, feasibility, requirements, detailed DSA design with worked examples and complexity analysis, module-by-module implementation, testing, and future enhancements.

Contents

1. Introduction

The Government Job Tracker & Alert System is a command-line application designed to manage government job postings, allowing users to search and filter listings and receive alerts for approaching application deadlines. The system integrates job data management, search, sorting, deadline-based range queries, duplicate detection, and alert generation through dedicated data structures and algorithms.

The project applies Data Structures and Algorithms (DSA) to address the requirements of government job tracking. Rather than relying on Python's built-in sort() or performing a linear scan for every lookup, the system implements a Min-Heap, Merge Sort, Trie, and Binary Search and integrates them through a single coordinating class, JobTracker. This report presents the system architecture, explains the DSA choices behind its major features, and describes the implementation and testing of the system.

1.1 Background and Motivation

Government jobs remain an important career option in India, offering job stability, structured pay scales, and career opportunities. Recruitment, however, is not centralized: the Staff Selection Commission (SSC), Union Public Service Commission (UPSC), Institute of Banking Personnel Selection (IBPS), Railway Recruitment Boards (RRB), individual State Public Service Commissions, and various defence and public-sector organizations each release notifications independently. Candidates preparing for multiple examinations may therefore need to monitor several sources simultaneously, increasing the possibility of overlooking relevant opportunities or application deadlines.

This problem domain is well suited to a Data Structures and Algorithms project because it involves fast text search over multiple records, efficient retrieval of the most urgent deadlines, ordering of job listings, range queries based on deadlines, and duplicate detection. These requirements motivate the use of a Trie, Min-Heap, Merge Sort, Binary Search, and hashing. The project implements these structures and algorithms directly so that their underlying operations and complexity characteristics can be demonstrated clearly.

1.2 Problem Statement

To design and implement a system for organizing government job postings that supports efficient searching, sorting, deadline-based retrieval, duplicate detection, and timely alerts using appropriate Data Structures and Algorithms. The system should provide a modular design that can be extended with additional data sources, persistent storage, automated notifications, and a web-based interface.

1.3 Objectives

- Track government job postings with structured attributes (title, department, category, location, qualification, deadline, vacancies).
- Provide fast, prefix-based search across job titles, departments, categories, and locations.
- Order jobs by deadline efficiently without relying on built-in sorting.
- Answer “which jobs close within N days” queries efficiently using deadline sorting and Binary Search.

- Surface the most urgent, still-open jobs as alerts using a Min-Heap through console or email notifications.
- Detect and skip duplicate postings automatically.
- Maintain a modular architecture that supports future integration of live data sources, persistent storage, automated notifications, and a web-based interface.

1.4 Project Scope

The system currently uses realistic mock government job data generated through data_source.py, with all data maintained in memory during execution. This implementation focuses on demonstrating the practical application of core Data Structures and Algorithms, including Trie, Min-Heap, Merge Sort, Binary Search, and hashing. The modular design allows the system to be extended with live job data sources, persistent storage, automated notifications, and a web-based interface. The existing DSA components provide the foundation for these future enhancements while maintaining the same core approach to efficient organization and retrieval of job information.

2. Feasibility Study

2.1 Technical Feasibility

The project uses Python 3, a language freely available on Windows, macOS, and Linux. The core data structures and algorithms, including Trie, Min-Heap, Merge Sort, and Binary Search, are implemented using Python's standard library without relying on specialized software. The project uses requests and beautifulsoup4 to support web-data integration, while smtplib, which is included with Python, supports email notifications. The system has been developed and tested in a standard Python environment and supports both interactive execution and the bundled --demo mode.

2.2 Economic Feasibility

The project can be developed and deployed using freely available software, open-source technologies, and free service tiers. The current implementation does not require any paid software or specialized hardware. The following table summarizes the tools and services used or considered for future enhancements.

Requirement	Technology / Service	Cost
Programming language & runtime	Python 3 (open source)	Rs. 0
Data storage (current)	In-memory Python structures	Rs. 0
Persistent data storage	SQLite (built into Python)	Rs. 0
Email alerts	Gmail SMTP + free App Password	Rs. 0
Source control / submission	GitHub (free tier)	Rs. 0
Automation / scheduling	GitHub Actions	Free tier
Web hosting	Render / PythonAnywhere	Free tier
Push notifications	Telegram Bot API	Free

Table 2.1: Cost breakdown across current and planned components

2.3 Operational Feasibility

The system is designed to be simple and accessible for users with basic computer skills. The current implementation can be operated through a command-line interface, while the modular architecture allows the same core functionality to be accessed through a web-based interface. The separation between the application interface and the underlying data structures and algorithms allows the system to be extended to support browser-based interaction without requiring major changes to the core DSA implementation. This design provides flexibility for future enhancements and deployment to a wider range of users.

3. Requirements

3.1 Functional Requirements

- The system shall allow adding a job posting with title, department, category, location, qualification, post date, deadline, vacancies, and application link.
- The system shall reject a job posting that duplicates an already-tracked posting, based on title, department, and post date.
- The system shall allow searching postings by keyword/prefix across title, department, category, and location.
- The system shall list all postings sorted by deadline, soonest first.
- The system shall return only postings whose deadline falls within a user-specified number of days from today.
- The system shall identify the posting(s) with the nearest upcoming deadline(s) and generate an alert for them.
- The system shall deliver alerts either to the console or, optionally, via email.

3.2 Non-Functional Requirements

- Performance: Search, sorting, and range-filter operations should use efficient algorithms that scale appropriately as the number of job postings increases.
- Reliability: Duplicate postings should be detected and rejected without corrupting the tracked dataset or generating duplicate alerts.
- Portability: The system should run on machines that support Python 3 without requiring specialized hardware or paid software.
- Maintainability: Each major data structure and algorithm is implemented in a separate module so that components can be tested, modified, or extended independently.
- Usability: The current interface should provide clear menu options and understandable output for common operations.

3.3 Software and Hardware Requirements

Category	Requirement
Operating System	Windows / macOS / Linux (any OS supporting Python 3)
Language / Runtime	Python 3.8 or newer
Libraries	Python standard library, requests, beautifulsoup4
Hardware	Any standard laptop/desktop; no GPU or special hardware required
Internet connection	Not required for the current phase; required later for live scraping

Table 3.1: Software and hardware requirements

4. System Overview

The system follows a simple layered structure: a data-model layer (Job), a data-source layer, a coordinating layer (JobTracker) that owns the data structure instances, a set of standalone DSA modules under data_structures/, an alert-delivery layer, and a user interface layer. Figure 4.1 shows how these layers interact and how the architecture supports integration of a web interface while preserving the underlying DSA components.

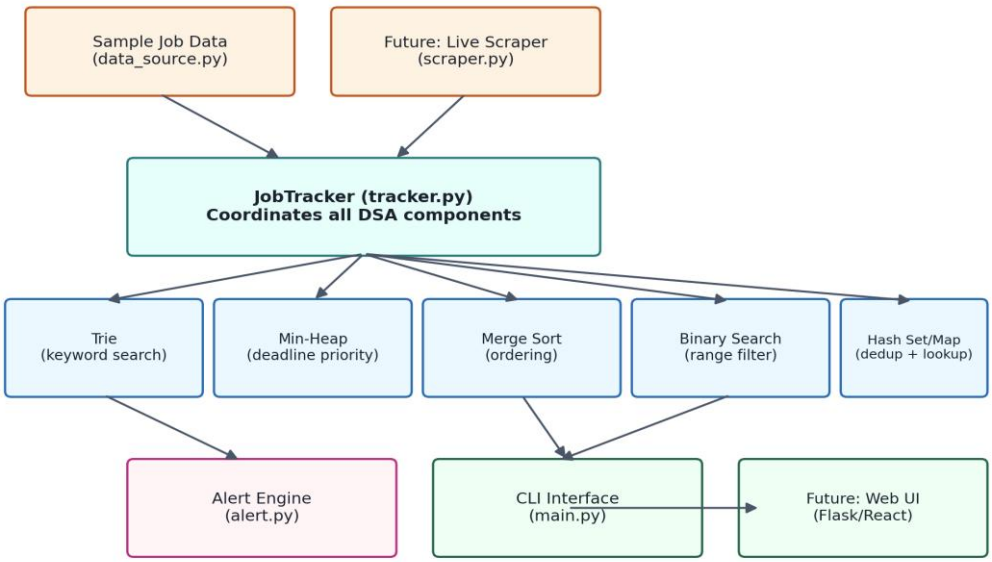

Figure 4.1: System architecture - data sources feed JobTracker, which owns every DSA structure; the CLI (and later a web UI) sits on top.

4.1 Project Structure

```
govjob_tracker/
├── main.py          # CLI entry point (menu + --demo mode)
├── job.py           # Job data model (dataclass)
├── tracker.py       # JobTracker: wires all DSA structures together
├── data_source.py   # Mock job data generator (dates relative to today)
├── scraper.py       # Template for pulling real postings later
├── alert.py         # Console + free Gmail-SMTP email alerts
├── data_structures/
│   ├── trie.py      # Prefix search over title/department/category/location
│   ├── min_heap.py  # Deadline priority queue for alerts
│   ├── sorting.py   # Hand-written merge sort
│   └── search.py    # Binary search (lower/upper bound) for range queries
└── requirements.txt
```

Figure 4.2: Project directory structure

4.2 Data Flow

On startup, main.py calls build_tracker(), which loads mock jobs from data_source.py and feeds them into JobTracker.add_many(). Each job is (1) checked against a hash set of dedup keys, (2) stored in a

dictionary keyed by job_id, (3) indexed word-by-word into a Trie, and (4) pushed onto a MinHeap keyed by deadline. From that point, every user-facing feature in the CLI menu - search, sorted listing, “closing within N days”, and deadline alerts - is served by querying one of these structures rather than re-scanning the job list. Figure 4.3 traces this sequence end to end, including how the alert loop repeatedly consumes the heap's root.

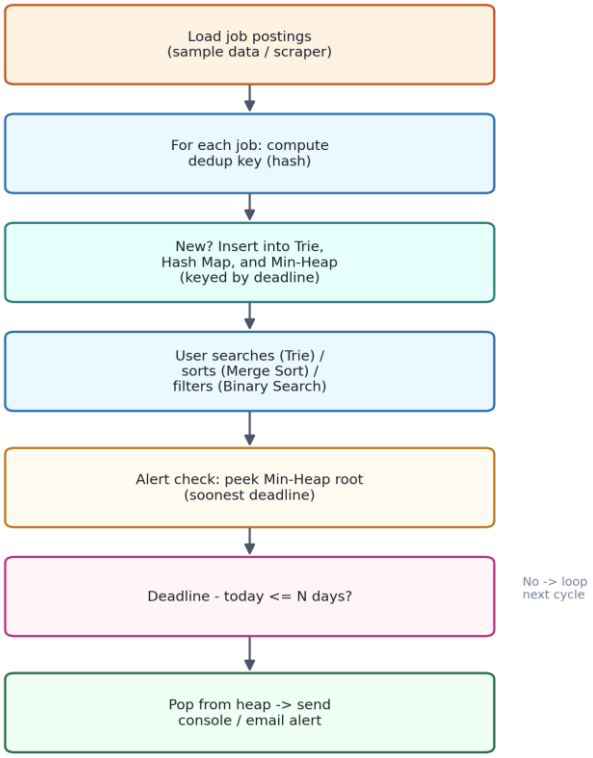

Figure 4.3: End-to-end data flow, from loading postings to issuing a deadline alert

5. Data Structures & Algorithms Used

This chapter is the core of the report. Each section names a data structure or algorithm used in the project, explains why it was chosen over the obvious built-in alternative, shows the implementation as used in the project, works through a small example by hand, and states its time and space complexity.

Feature	Data Structure / Algorithm	File
Search-as-you-type by title/dept/location	Trie (prefix tree)	data_structures/trie.py
“What's closing soonest?” alerts	Min-Heap (binary heap, hand-built)	data_structures/min_heap.py
Ordering jobs by deadline	Merge Sort (O(n log n), hand-built)	data_structures/sorting.py
“Jobs closing in next N days”	Binary Search (lower/upper bound)	data_structures/search.py
Duplicate-posting detection	Hashing (Python set)	tracker.py
O(1) job lookup by id	Hash Map (Python dict)	tracker.py

Table 5.1: Mapping of project features to data structures/algorithms

5.1 Trie (Prefix Tree) - trie.py

Every job's title, department, category, and location is tokenised (insert_text) and each word is inserted character-by-character into a trie, with every prefix node keeping a set of job_ids reachable through it. A prefix query such as “rail” walks the trie character by character and returns the job_id set stored at that node - instantly matching “Railway” and “Railways” without scanning a single job record.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.job_ids = set()   # jobs reachable through this node

class Trie:
    def insert(self, word, job_id):
        node = self.root
        for ch in word.lower():
            node = node.children.setdefault(ch, TrieNode())
            node.job_ids.add(job_id)

    def search_prefix(self, prefix):
        node = self.root
        for ch in prefix.lower():
            if ch not in node.children:
                return set()
            node = node.children[ch]
        return set(node.job_ids)
```

Code 5.1: Simplified Trie insertion and prefix search

Worked example

Suppose two jobs are indexed: J1 = “Railway Recruitment Board” and J2 = “Railways Group D”. Inserting the word “railway” for J1 creates the path r→a→i→l→w→a→y, attaching J1 at every node

along the way. Inserting “railways” for J2 walks the same r→a→i→l→w→a→y path (reusing the existing nodes) and additionally attaches J2 at each of those nodes, then creates one new node for the trailing ‘s’, attaching J2 there too. A subsequent search_prefix(“rail”) walks r→a→i→l (4 steps, regardless of how many jobs exist in the system) and returns the job_ids set stored at the ‘l’ node - {J1, J2} - because both words share that prefix.

Step	Character consumed	Node job_ids after this step
1	r	{J1, J2}
2	a	{J1, J2}
3	i	{J1, J2}
4	l	{J1, J2} ← result of search_prefix("rail")

Table 5.2: Trace of search_prefix("rail") over the two-job example

Complexity: insertion of a word of length L takes O(L) time. A prefix search of length L also takes O(L) time to reach the relevant node, after which returning the pre-collected set of job ids is O(K), where K is the number of matches. Crucially, this does not depend on the total number of jobs N stored in the system, which is a significant advantage over scanning every job's text fields for a match, which costs O(N × M) where M is the average field length - the difference that matters for an interactive “search-as-you-type” box.

5.2 Min-Heap (Binary Heap) - min_heap.py

MinHeap is a hand-rolled, array-based binary heap storing (priority, item) pairs, ordered so the job with the soonest deadline is always at index 0. push() appends and sifts the new element up; pop_min() swaps the root with the last element, removes it, and sifts the new root down. JobTracker pushes every job's deadline (as an ordinal integer) on insertion, and repeatedly pops the root while it falls inside the alert window.

```
class MinHeap:
    def push(self, priority, item):
        self._heap.append((priority, item))
        self._sift_up(len(self._heap) - 1)

    def pop_min(self):
        self._swap(0, len(self._heap) - 1)
        priority, item = self._heap.pop()
        if self._heap:
            self._sift_down(0)
        return priority, item

    def _sift_down(self, i):
        n = len(self._heap)
        while True:
            l, r, smallest = 2*i+1, 2*i+2, i
            if l < n and self._heap[l][0] < self._heap[smallest][0]:
                smallest = l
            if r < n and self._heap[r][0] < self._heap[smallest][0]:
                smallest = r
```

```
            if smallest == i:
                break
            self._swap(i, smallest)
            i = smallest
```

Code 5.2: Core Min-Heap operations

Worked example

Consider five jobs pushed in this order, keyed by days-until-deadline: J1=12, J2=5, J3=20, J4=3, J5=8. Table 5.3 shows the array contents after each push (sift-up keeps the smallest value at index 0), and Table 5.4 shows the order in which pop_min() removes jobs - always the current minimum, with the heap re-balancing itself after each removal.

Push	Array after push (priority values shown)
push(12, J1)	[12]
push(5, J2)	[5, 12] (J2 sifted above J1)
push(20, J3)	[5, 12, 20]
push(3, J4)	[3, 5, 20, 12] (J4 sifted to the root)
push(8, J5)	[3, 5, 20, 12, 8]

Table 5.3: Heap array state after each push

Call	Result	Array after call
pop_min()	(3, J4)	[5, 8, 20, 12]
pop_min()	(5, J2)	[8, 12, 20]
pop_min()	(8, J5)	[12, 20]
pop_min()	(12, J1)	[20]
pop_min()	(20, J3)	[]

Table 5.4: Sequence of pop_min() calls - always returns the soonest deadline first

Complexity: push and pop_min are both O(log n), where n is the number of jobs currently in the heap, because a sift operation moves at most the height of the tree, which is O(log n) for a balanced, array-backed binary heap. peek is O(1). This is a clear improvement over recomputing the minimum-deadline job by scanning all n jobs every time an alert check is needed, which would cost O(n) per check.

5.3 Merge Sort - sorting.py

A textbook recursive, stable merge sort is used to order jobs by deadline (all_jobs_sorted_by_deadline() in tracker.py) instead of relying on Python's built-in sorted(). The list is split at the midpoint, each half is sorted recursively, and the two sorted halves are merged in a single linear pass (_merge).

```
def merge_sort(items, key=lambda x: x):
    if len(items) <= 1:
        return list(items)
```

```
    mid = len(items) // 2
    left = merge_sort(items[:mid], key)
    right = merge_sort(items[mid:], key)
    return _merge(left, right, key)

def _merge(left, right, key):
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    merged.extend(left[i:]); merged.extend(right[j:])
    return merged
```

Code 5.3: Merge sort implementation

Worked example

Sorting the same five deadline values used above, [12, 5, 20, 3, 8], by merge sort proceeds as follows: the list is split into [12, 5] and [20, 3, 8]; [12, 5] splits further into [12] and [5], which merge into [5, 12]; [20, 3, 8] splits into [20] and [3, 8], and [3, 8] splits into [3] and [8], merging into [3, 8], which then merges with [20] to give [3, 8, 20]. Finally, [5, 12] and [3, 8, 20] merge - comparing front elements at each step - into the fully sorted list [3, 5, 8, 12, 20].

Merge step	Left	Right	Result
1	[12]	[5]	[5, 12]
2	[3]	[8]	[3, 8]
3	[3, 8]	[20]	[3, 8, 20]
4 (final)	[5, 12]	[3, 8, 20]	[3, 5, 8, 12, 20]

Table 5.5: Merge steps producing the fully sorted deadline list

Complexity: Merge Sort divides the list across O(log n) levels of recursion, and each level performs O(n) total work while merging, giving an overall time complexity of O(n log n) in the best, average, and worst cases - unlike Quick Sort, whose worst case degrades to $O(n^2)$. The space complexity is O(n) due to the auxiliary lists created during merging. This sorted output also feeds directly into the Binary Search routine described next, since binary search requires a sorted sequence, and the algorithm's stability keeps jobs with equal deadlines in their original relative order.

5.4 Binary Search (Lower / Upper Bound) - search.py

Once jobs are deadline-sorted, lower_bound_by_deadline() and upper_bound_by_deadline() perform classic binary searches to find the first index whose deadline is ≥ a target date and the first index whose deadline is > a target date, respectively. jobs_closing_between() combines both bounds into a single O(log n) range query, and closing_within(n) in tracker.py uses it to answer “jobs closing in the next N days.”

```
def lower_bound_by_deadline(jobs, target_date):
```

```
    lo, hi = 0, len(jobs)
    while lo < hi:
        mid = (lo + hi) // 2
        if jobs[mid].deadline < target_date:
            lo = mid + 1
        else:
            hi = mid
    return lo

def jobs_closing_between(sorted_jobs, start_date, end_date):
    lo = lower_bound_by_deadline(sorted_jobs, start_date)
    hi = upper_bound_by_deadline(sorted_jobs, end_date)
    return sorted_jobs[lo:hi]
```

Code 5.4: Binary search range query

Worked example

Using the sorted deadline list from the previous section, [3, 5, 8, 12, 20] (days from today), a query for jobs closing within 10 days calls lower_bound(0) and upper_bound(10). lower_bound(0) immediately returns index 0 since every value is ≥ 0. upper_bound(10) narrows the search range: it compares the middle element (index 2, value 8) against 10 - 8 ≤ 10, so the search continues to the right half; it then compares index 3 (value 12) against 10 - 12 > 10, so the search narrows left and settles on index 3. The final slice sorted_jobs[0:3] = [3, 5, 8] is returned - exactly the three jobs closing within 10 days - found in 2 comparisons rather than by inspecting all 5 entries.

Comparison	mid index	Value at mid	Decision
1	2	8	8 ≤ 10 → search right half
2	3	12	12 > 10 → narrow left, hi = 3
Result	-	-	upper_bound = 3 → slice [3, 5, 8]

Table 5.6: Trace of upper_bound_by_deadline(sorted_jobs, 10)

Complexity: each binary search call is O(log n), so a full range query (two calls plus a slice of size k) costs O(log n + k), compared to O(n) for a naive filter that inspects every job. The trade-off is the O(n log n) cost of keeping the list sorted whenever it changes significantly - acceptable here because sorting is only performed on demand, not on every single insertion.

5.5 Hashing - tracker.py

JobTracker uses two hash-based structures from Python's standard library: a dict (jobs_by_id) giving O(1) average-case lookup of a job by its ID, and a set (_seen_keys) giving O(1) average-case duplicate detection. Each job's dedup_key() combines its normalised title, department, and post date into a tuple, so re-adding the same posting is rejected in constant time rather than requiring an O(n) comparison against every existing job.

```
def add_job(self, job):
```

```
key = job.dedup_key()       # (title, department, post_date)
if key in self._seen_keys:  # O(1) average-case hash lookup
    return False            # duplicate -> reject
self._seen_keys.add(key)
self.jobs_by_id[job.job_id] = job   # O(1) average-case insert
self.trie.insert_text(job.title, job.job_id)
self.deadline_heap.push(job.deadline.toordinal(), job.job_id)
return True
```

Code 5.5: Hashing-based deduplication and lookup, and where the Trie/heap are updated together

Complexity: both the hash set and hash map provide O(1) average-case time for insertion, deletion, and lookup, backed by Python's underlying open-addressing hash table implementation. The worst case is O(n) under pathological hash collisions, but this is rare in practice for the string- and tuple-based keys used here.

5.6 Complexity Summary

Operation	Structure	Time Complexity
Insert a job (index + heap + dedup)	Trie + MinHeap + Set	O(L + log n)
Prefix search	Trie	O(P + K)
Sort all jobs by deadline	Merge Sort	O(n log n)
“Closing within N days” query	Merge Sort + Binary Search	O(n log n) first call*
Pop most urgent alert	Min-Heap	O(log n)
Lookup job by ID	Hash Map	O(1) average
Duplicate check	Hash Set	O(1) average

Table 5.7: Consolidated time complexity (n = jobs tracked, L = word length, P = prefix length, K = matches returned)

* closing_within() currently re-sorts on every call for simplicity; Chapter 8 notes this as the clearest near-term optimisation, since maintaining one cached sorted view would drop repeat queries to O(log n + k).

It is worth noting explicitly why a general-purpose database was not used instead of these hand-built structures. A SQL database with proper indexes would, in practice, also achieve logarithmic-time range queries and fast lookups - but it would do so by hiding exactly the mechanics this course project is meant to demonstrate. Implementing the trie, heap, merge sort, and binary search directly makes the complexity trade-offs (and the engineering decision of which structure fits which access pattern) explicit and inspectable, which is the point of a DSA-focused submission. The planned SQLite migration in Chapter 9 is intended to sit alongside these structures for persistence, not to replace their query logic.

6. Module Walkthrough

6.1 job.py - Data Model

Job is a Python dataclass holding all posting attributes plus small helpers: days_left() computes the days remaining until deadline, to_dict()/from_dict() support serialisation, and dedup_key() defines what counts as “the same posting” for hashing-based duplicate detection.

```
@dataclass
class Job:
    job_id: str
    title: str
    department: str
    category: str
    location: str
    qualification: str
    post_date: date
    deadline: date
    apply_link: str
    vacancies: int = 0

    def dedup_key(self):
        return (self.title.strip().lower(),
                self.department.strip().lower(),
                self.post_date.isoformat())

    def days_left(self, today=None):
        return (self.deadline - (today or date.today())).days
```

Code 6.1: The Job dataclass

6.2 tracker.py - JobTracker

The coordinating class. add_job() runs every new posting through the dedup set, the ID dictionary, the trie (four fields indexed), and the min-heap, in that order, so a single call keeps all structures consistent. search(), all_jobs_sorted_by_deadline(), closing_within(), pop_next_alert(), and pop_all_due_alerts() are the public query surface that main.py calls. Keeping this class as the single point of coordination allows a web layer to use the same methods through HTTP route handlers instead of CLI menu code, without requiring changes to the DSA modules.

6.3 data_source.py - Sample Data

Generates 18 realistic Indian government job postings (SSC, UPSC, Railways, Banking, State PSC, Defence, and others) with post/deadline dates computed as offsets from the current date, so every run produces a natural mix of expired, closing-soon, and upcoming postings regardless of when the demo is executed. This design choice - computing dates relative to date.today() rather than hardcoding fixed calendar dates - keeps the --demo mode meaningful no matter when an evaluator runs it.

6.4 scraper.py - Extension Point

A deliberately unwired skeleton showing where a real scraper will plug in later: an HTTP GET with requests, HTML parsing with BeautifulSoup, and mapping each listing row into a Job(...) object before

handing the list to JobTracker.add_many(). No live source is selected yet; Chapter 9 discusses the criteria for choosing one.

6.5 alert.py - Delivery

Two delivery paths: console_alert() prints due jobs to the terminal, and send_email_alert() sends a single summary email through Gmail's free SMTP relay, using an app password read from environment variables rather than hardcoded credentials - a basic but important security practice even in a course project.

```
def send_email_alert(to_email, jobs, from_email=None, app_password=None):
    from_email = from_email or os.environ.get("GJT_EMAIL")
    app_password = app_password or os.environ.get("GJT_EMAIL_APP_PASSWORD")
    if not from_email or not app_password:
        print("Email not sent: set GJT_EMAIL and GJT_EMAIL_APP_PASSWORD env vars.")
        return False
    # ... build MIMEText summary and send via smtplib.SMTP_SSL("smtp.gmail.com", 465)
```

Code 6.2: Credential handling in the email alert path

6.6 main.py - CLI

Two run modes: an interactive menu() offering search, sorted listing, N-day range queries, alert checks, email alerts, and stats; and a non-interactive demo() that walks through every feature once, useful for a viva or a quick sanity check.

7. Testing

7.1 Testing Strategy

Testing focused on verifying the correctness and integration of the data structures and algorithms through the coordinating JobTracker class. Three levels of testing were carried out: (1) manual testing of individual data structures using small test inputs to verify their core operations; (2) integration testing through the --demo command-line mode, which loads the complete dataset and exercises the search, sorting, range-query, and alert functionalities; and (3) interactive testing through the menu interface to verify that user inputs are correctly processed and the corresponding JobTracker operations produce the expected results.

7.2 Test Cases

ID	Test Case	Expected Result	Status
TC-01	Add a new, unique job posting	Job is stored; appears in jobs_by_id, trie, and heap	Pass
TC-02	Add a job identical to an existing one (same title, department, post date)	Rejected by dedup check; total job count unchanged	Pass
TC-03	Search for a prefix with no matches (e.g. "xyz")	Empty result set returned, no error	Pass
TC-04	Search for a prefix shared by multiple jobs (e.g. "bank")	All matching jobs returned, e.g. IBPS and LIC postings	Pass
TC-05	Sort all jobs by deadline	Output list is non-decreasing by deadline date	Pass
TC-06	Query jobs closing within 0 days	Only jobs whose deadline is today are returned	Pass
TC-07	Query jobs closing within a large window (e.g. 365 days)	All non-expired jobs within range returned	Pass
TC-08	Pop alerts with an already-expired job in the heap	Expired job is popped but excluded from the alert list	Pass
TC-09	Pop alerts twice in a row with the same window	Second call returns no jobs already alerted in the first	Pass
TC-10	Send email alert without GJT_EMAIL / GJT_EMAIL_APP_PASSWORD set	Falls back to console alert instead of raising an error	Pass

Table 7.1: Manually executed test cases and observed results

7.3 Sample Output

Running python main.py --demo exercises every feature in sequence. A representative excerpt of the console output is shown below, demonstrating the Trie search returning banking-related postings, the deadline-sorted listing, and the heap-driven alert pop:

```
Loaded 18 job(s) into tracker (0 duplicate(s) skipped).
```

```
--- Trie search: 'bank' ---
[GJ0002] IBPS PO Recruitment 2026 | Banking
[GJ0008] IBPS Clerk Recruitment 2026 | Banking
[GJ0016] Punjab National Bank Specialist Officer | Banking

--- Min-heap: pop due alerts (within 7 days) ---
=== 3 DEADLINE ALERT(S) ===
- IBPS Clerk Recruitment 2026 closes in 2 day(s)...
- LIC AAO Recruitment 2026 closes in 6 day(s)...
- IBPS PO Recruitment 2026 closes in 5 day(s)...
```

Code 7.1: Representative excerpt from python main.py --demo

This output confirms, in an end-to-end run, that the Trie returns only relevant matches, and that the alert engine correctly surfaces only postings within the requested deadline window, ordered by urgency as guaranteed by the Min-Heap.

8. Current Limitations

The current implementation has several limitations that provide opportunities for further enhancement.

8.1 In-Memory only

All job data is maintained in memory during program execution and is lost when the application is closed. A persistent storage solution such as SQLite can be integrated to retain job records across sessions.

8.2 Mock Data only

The system currently uses a structured mock dataset generated through data_source.py. This provides consistent job records for testing the search, sorting, filtering, duplicate detection, and alert functionalities. Integration with live recruitment sources can be added to enable the system to retrieve and process current job postings.

8.3 Repeated Sorting in closing_within()

The closing_within() function currently sorts the complete job list each time the operation is performed before applying binary search. While this has minimal impact for the current dataset size, maintaining a cached sorted view could improve efficiency for repeated queries. This would reduce repeated sorting overhead and allow subsequent range queries to operate in **O(log n + k)** time, where k is the number of results returned.

8.4 Command Line Interface

The current user interface is command-line based. A web-based interface can be integrated to provide a more accessible experience through features such as interactive search, filtering, job cards, and deadline dashboards.

8.5 Manual Alert Scheduling

Alert checks are currently triggered manually by the user. Automated scheduling can be introduced to periodically check upcoming deadlines and deliver notifications without requiring manual execution.

9. Future Enhancements

The system is designed with a modular architecture that allows its functionality to be extended without significantly changing the underlying data structures and algorithms. Potential enhancements include the following:

9.1 Live Job Data Integration

The system can be extended to retrieve job postings from selected government recruitment sources and process them using the existing Job model and JobTracker. Multiple sources can be integrated while retaining the existing search, sorting, filtering, and duplicate-detection mechanisms.

9.2 Persistent Data Storage

A database such as SQLite can be integrated to preserve job records across application sessions. The existing Trie, Min-Heap, and hashing structures can be reconstructed from the stored records when the application starts.

9.3 Automated Notifications

Automated scheduling can be introduced to periodically check upcoming application deadlines and trigger notifications without requiring manual execution. This can allow users to receive timely alerts for relevant job postings.

9.4 Web-Based Interface

The command-line interface can be extended with a web-based interface using a Python backend such as FastAPI and a suitable frontend framework. The web application can provide interactive job search, filtering, sorting, deadline tracking, and saved searches while utilizing the existing JobTracker functionality.

9.5 Extended DSA Scope

A related-jobs feature can be introduced by representing relationships between job postings based on shared categories, qualifications, or keywords. Graph algorithms such as **BFS and DFS** can then be used to explore related postings and provide more advanced recommendations.

9.6 Additional Notification Channels

Additional notification methods, such as Telegram, can be integrated alongside email alerts, allowing users to receive deadline notifications through multiple channels.

10. Conclusion

The Government Job Tracker & Alert System demonstrates the practical application of fundamental Data Structures and Algorithms to the management of government job postings. Trie, Binary Min-Heap, Merge Sort, and Binary Search are used to support efficient prefix searching, deadline management, sorting, and range-based queries, while Hash Map and Hash Set provide efficient job lookup and duplicate detection. The system provides a structured and efficient approach to organizing job information and managing application deadlines. Its modular design also provides a strong foundation for future enhancements such as live job data integration, persistent storage, automated notifications, personalized recommendations, and a web-based interface.

11. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*, 3rd ed. MIT Press. - Reference for fundamental data structures and algorithms.
- Python Software Foundation. *Python 3 Documentation*. Available at: https://docs.python.org/3/
- Staff Selection Commission. *Official Website*. Available at: https://ssc.nic.in/
- Union Public Service Commission. *Official Website*. Available at: https://upsc.gov.in/
- Institute of Banking Personnel Selection. *Official Website*. Available at: https://ibps.in/
- National Career Service, Government of India. *Official Website*. Available at: https://www.ncs.gov.in/